

0.25.2. Le carré, côté par côté — doublé, deux positions

Les deux carrés : le A à gauche, puis le B à droite.

1. Ce qu'il faut comprendre

C'est le 0.25 et le 0.25.1 réunis : le carré du A, puis celui du B.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_x>` → [0.17.3. La fonction `deplace\_x\(\)` — avancer sur une ligne](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <deplace_y>` → [0.20.3. La fonction `deplace\_y\(\)` — avancer sur une colonne](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1394 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      73 o ← la console, à l'allumage – la mise en route de la console
deplace_x()    158 o ← main() – une fonction fournie par le langage
deplace_y()    158 o ← main() – une fonction fournie par le langage
main()         183 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() – efface des cases
CopierTuiles   18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()      x 4    52 o sur place, dans deplace_x(), deplace_y() + AdresseCase (10 o)
effacer()    x 2    30 o sur place, dans deplace_x(), deplace_y() + AdresseCase (10 o), EffacerCases (12 o)
image()      x 3     9 o sur place, dans deplace_x(), deplace_y(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
deplace_x() x 4    80 o sur place, dans main()
deplace_y() x 4    80 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_x> // fait avancer une tuile sur sa ligne
#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A
#include <deplace_y> // fait avancer une tuile sur sa colonne
```

2. UNE VARIABLE — PAS

```
uint8_t pas = 5;      // 5 cases, sur chaque axe
```

3. UNE VARIABLE — X

```
uint8_t x = 0;        // la colonne du A
```

4. UNE VARIABLE — Y

```
uint8_t y = 0;        // sa ligne
```

5. UNE VARIABLE — XB

```
uint8_t xb = 10;      // la colonne du B
```

6. UNE VARIABLE — YB

```
uint8_t yb = 0;        // sa ligne
                        // Deux lettres : chacune a SES deux variables.
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {           // Le programme commence ici.
    // 1. Le carré du A (ALPHABET[0]), depuis (0, 0) :
    deplace_x(x, y, ALPHABET[0], pas);    // à droite : x vaut 5
    deplace_y(x, y, ALPHABET[0], pas);    // en bas   : y vaut 5
    deplace_x(x, y, ALPHABET[0], -pas);   // à gauche : x revient à 0
    deplace_y(x, y, ALPHABET[0], -pas);   // en haut  : y revient à 0
    // 2. Puis le carré du B (ALPHABET[1]), depuis (10, 0), avec xb et yb :
    deplace_x(xb, yb, ALPHABET[1], pas);  // à droite : xb vaut 15
    deplace_y(xb, yb, ALPHABET[1], pas);  // en bas   : yb vaut 5
    deplace_x(xb, yb, ALPHABET[1], -pas); // à gauche : xb revient à 10
    deplace_y(xb, yb, ALPHABET[1], -pas); // en haut  : yb revient à 0

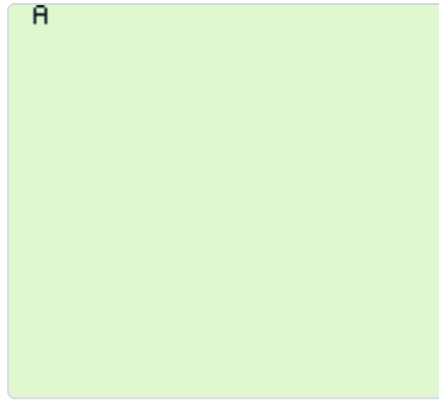
    while (true) {      // La boucle de jeu, pour toujours :
        image();        // attendre l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

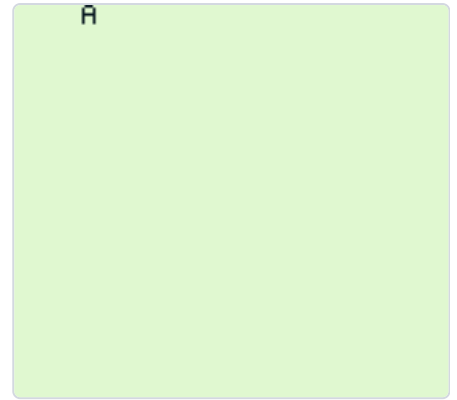
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



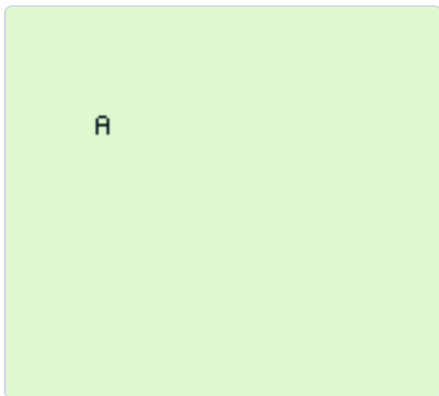
la toute première image — la console vient de démarrer — image 1



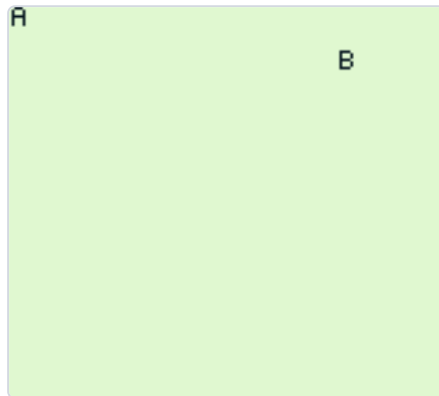
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A fait son carré à gauche, puis le B à droite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A passe par (5, 5)

✓ le A finit en (0, 0) (0,0)

✓ le B passe par (15, 5)

✓ le B finit en (10, 0) (10,0)

Ensuite : **0.25.3. La fonction `deplace()` — avancer sur les deux axes** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++